

p0571r1

Type Requirements for `<numeric>` Algorithms

Draft Proposal, 2 February 2017

This version:

<http://wg21.link/P0571r1>

Author:

[Bryce Adelstein Lelbach](#) (Lawrence Berkeley National Laboratory)

Audience:

SG1, LEWG, LWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Table of Contents

1	Overview
2	Kinds of Algorithms in <code><numeric></code>
2.1	In-Order-Accumulator Algorithms
2.2	<code>GENERALIZED_NONCOMMUTATIVE_SUM</code> and <code>GENERALIZED_SUM</code> Algorithms
3	What Should the Intermediate Type Be?
3.1	Intermediate Type for Ordered <code><numeric></code> Algorithms with Initial Value Parameters
3.2	Intermediate Type for Ordered <code><numeric></code> Algorithms without Initial Value Parameters
3.3	Intermediate Type for Unordered <code><numeric></code> Algorithms
3.4	Intermediate Type Policy for <code><numeric></code> Algorithms
4	Type Requirements
5	Proposed Changes
	References
	Informative References

§ 1. Overview

At the Issaquah 2016 meeting, during the LWG review of [\[P0452r0\]](#), it was realized that some of the numeric algorithms (both old algorithms and new ones from the Parallelism TS) had insufficient or unclear type requirements. This paper identifies some of the issues and proposes potential solutions.

[This chart](#) provides an overview of the current state of the type requirements of the `<numeric>` algorithms.

This proposal is not intended to change the existing design, specify any previous unspecified behavior which major implementations do not already conform to, or remove functionality. It simply clarifies and improves the specification of the `<numeric>` algorithms.

§ 2. Kinds of Algorithms in `<numeric>`

There are three kinds of algorithms in `<numeric>`:

- `accumulate`, `inner_product`, `partial_sum` and `adjacent_difference` have "in-order-accumulator" style wording.
- `exclusive_scan`, `inclusive_scan`, `transform_exclusive_scan`, `transform_inclusive_scan` are specified using `GENERALIZED_NONCOMMUTATIVE_SUM`.
- `reduce`, `transform_reduce` are specified via `GENERALIZED_SUM`.

§ 2.1. In-Order-Accumulator Algorithms

The wording for all of these algorithms fits the following pattern:

- Create an `acc` object which is initialized with `init` if the algorithm signature contains an initial value parameter and `*first` otherwise.
- For (the) iterator(s) in the range(s) **in order**, modify `acc` by applying a binary update operation which takes `acc` and the dereferenced value(s) of the iterator(s) as arguments: e.g. `acc = binary_op(acc, *i)`.

The ordering requirement is necessary to ensure that these algorithms are well-defined for **non-associative** and/or **non-commutative** operations, such as floating-point addition and multiplication (non-associative and commutative), and subtraction (non-associative and non-commutative).

Currently, the wording forces each iteration to depend on the prior iteration to ensure the correct ordering of accumulation. This introduces a loop-carried dependency that makes it impossible to parallelize.

§ 2.2. `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` Algorithms

To parallelize these operations, we need to be able to re-order applications of the operations, partition the workload into sub-tasks and then combine the results of the sub-tasks together using the operator.

This, however, would give a non-deterministic result for non-associative or non-commutative operations; for example, floating point arithmetic.

In addition to adding entirely new `<numeric>` algorithms, the Parallelism TS introduced new algorithms which perform the same operations as `accumulate`, `inner_product` and `partial_sum`, but have weaker constraints that allow parallelization:

- `reduce` is a parallelizable variant of `accumulate` and `transform_reduce` is a parallelizable variant `inner_product`. They may produce non-deterministic results for non-associative or non-commutative operations.
- `inclusive_scan` is a parallelizable variant of `partial_sum`. It may produce non-deterministic results for non-associative operations, but is fine for non-commutative operations.

These `<numeric>` algorithms are specified using the `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` definitions:

26.2 Definitions [numerics.defns]

1 Define `GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aN)` as follows:

- `a1` when `N` is 1, otherwise
- `op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN))` for any `K` where `1 < K + 1 = M ≤ N`.

2 Define `GENERALIZED_SUM(op, a1, ..., aN)` as `GENERALIZED_NONCOMMUTATIVE_SUM(op, b1, ..., bN)` where `b1, ..., bN` may be any permutation of `a1, ..., aN`.

This definition allows:

- Arbitrary, nested partitioning of input elements for both `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`.
- Arbitrary reordering of input elements for `GENERALIZED_SUM`.

§ 3. What Should the Intermediate Type Be?

During computation of the final result, a `<numeric>` algorithm needs to store the result of accumulation thus far in temporary objects. The **intermediate type** is the type of these objects. Importantly, these temporary objects are passed as the first argument to the binary operator. For the accumulator-style `<numeric>` algorithms (`accumulate`, `inner_product`, `partial_sum`, `adjacent_difference`), the intermediate type is the type of the accumulator object `acc`.

The intermediate type is only clearly specified for 2 of the 10 `<numeric>` algorithms. Determining and specifying the intermediate type for these algorithms is our first step because the type requirements all revolve around the intermediate type.

§ 3.1. Intermediate Type for Ordered `<numeric>` Algorithms with Initial Value Parameters

`accumulate` and `inner_product` are described by the standard as:

26.8.2 Accumulate [`accumulate`]:

```
template <class InputIterator,
          class T>
    T accumulate(InputIterator first, InputIterator last,
                 T init);
template <class InputIterator,
          class T,
          class BinaryOperation>
    T accumulate(InputIterator first, InputIterator last,
                 T init,
                 BinaryOperation binary_op);
```

1 *Effects*: Computes its result by initializing the accumulator `acc` with the initial value `init` and then modifies it with `acc = acc + *i` or `acc = binary_op(acc, *i)` for every iterator `i` in the range `[first, last)` in order.

2 *Requires*: `T` shall meet the requirements of `CopyConstructible` (Table 22) and `CopyAssignable` (Table 24) types. In the range `[first, last]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.

26.8.5 Inner product [inner.product]:

```
template <class InputIterator1, class InputIterator2,
         class T>
    T inner_product(InputIterator1 first1, InputIterator1 last1,
                   InputIterator2 first2,
                   T init);
template <class InputIterator1, class InputIterator2,
         class T,
         class BinaryOperation1,
         class BinaryOperation2>
    T inner_product(InputIterator1 first1, InputIterator1 last1,
                   InputIterator2 first2,
                   T init,
                   BinaryOperation1 binary_op1,
                   BinaryOperation2 binary_op2);
```

1 *Effects*: Computes its result by initializing the accumulator `acc` with the initial value `init` and then modifying it with `acc = acc + (*i1) * (*i2)` or `acc = binary_op1(acc, binary_op2(*i1, *i2))` for every iterator `i1` in the range `[first1, last1)` and iterator `i2` in the range `[first2, first2 + (last1 - first1))` in order.

2 *Requires*: `T` shall meet the requirements of `CopyConstructible` (Table 22) and `CopyAssignable` (Table 24) types. In the ranges `[first1, last1]` and `[first2, first2 + (last1 - first1)]` `binary_op1` and `binary_op2` shall neither modify elements nor invalidate iterators or subranges.

This definition doesn't make it clear what the intermediate type is. Both algorithms have a requirement that the type of the initial value parameter (`T`) be `CopyConstructible` and `CopyAssignable`, implying that the accumulator object is intended to be of type `T`. Both `libc++` and `libstdc++` use `T` as the type of the accumulator object for these 2 algorithms.

Using `T` both has upsides:

```
vector<int> i{INT_MAX, INT_MAX};
big_int bi = accumulate(d.begin(), d.end(), big_int(0));
// big_int is an arbitrary-precision integer class which uses dynamic storage.

// bi == 2 * INT_MAX.
```

and downsides:

```
vector<double> d{0.5, 0.5};
double r = accumulate(d.begin(), d.end(), 0);

// r == 0, not 1. The accumulator's type was int, since the type of the literal
// 0 is int.
```

Alternative choices for the intermediate type are:

- Determine some common type between `T`, `InputIterator`'s value type, and the result of `binary_op` (or the relevant operator) and use that as the intermediate type. As discussed in [§3.2 Intermediate Type for Ordered <numeric> Algorithms without Initial Value Parameters](#), it may be difficult to do this if `binary_op` is overloaded in a certain manner and it may make it harder for users to determine what the intermediate type will be.
- Use the `InputIterator`'s value type as the intermediate type. This would make the example of accumulating a `vector<double>` produce a result that might be closer to what the user intended, but it would also remove the functionality desired by the user in the `big_int` example.

Switching to either of these alternatives would force implementations to make breaking changes, and neither option seems particularly attractive.

I suggest adopting the following design. It is simple and clear to both users and implementators:

`<numeric>` algorithms which take an initial value parameter and initial value type template parameter will use the initial value type as the intermediate type.

§ 3.2. Intermediate Type for Ordered `<numeric>` Algorithms without Initial Value Parameters

As we mentioned earlier, the type of the accumulator object (the intermediate type) is explicitly specified for only 2 of the 10 `<numeric>` algorithms: `partial_sum` and `adjacent_difference`:

26.8.6 Partial sum [`partial.sum`]

```
template <class InputIterator, class OutputIterator>
    OutputIterator partial_sum(InputIterator first, InputIterator last,
                              OutputIterator result);
template <class InputIterator, class OutputIterator,
          class BinaryOperation>
    OutputIterator partial_sum(InputIterator first, InputIterator last,
                              OutputIterator result,
                              BinaryOperation binary_op);
```

1 *Effects*: For a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, `acc` is then modified by `acc = acc + *i` or `acc = binary_op(acc, *i)` and the result is assigned to `*(result + (i - first))`.

2 *Returns*: `result + (last - first)`.

3 *Complexity*: Exactly `(last - first) - 1` applications of the binary operation.

4 *Requires*: `InputIterator`'s value type shall be constructible from the type of `*first`. The result of the expression `acc + *i` or `binary_op(acc, *i)` shall be implicitly convertible to `InputIterator`'s value type. `acc` shall be writable (24.2.1) to the `result` output iterator. In the ranges `[first, last]` and `[result, result + (last - first)]` `binary_op` shall neither modify elements nor invalidate iterators or subranges.

5 *Remarks*: `result` may be equal to `first`.

26.8.11 Adjacent difference [adjacent.difference]

```
template <class InputIterator, class OutputIterator>
    OutputIterator adjacent_difference(InputIterator first, InputIterator last,
                                      OutputIterator result);

template <class InputIterator, class OutputIterator,
          class BinaryOperation>
    OutputIterator adjacent_difference(InputIterator first, InputIterator last,
                                      OutputIterator result,
                                      BinaryOperation binary_op);
```

1 *Effects*: For a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, creates an object `val` whose type is `InputIterator`'s value type, initializes it with `*i`, computes `val - acc` or `binary_op(val, acc)`, assigns the result to `*(result + (i - first))`, and move assigns from `val` to `acc`.

2 *Requires*: `InputIterator`'s value type shall be `MoveAssignable` (Table 23) and shall be constructible from the type of `*first`. `acc` shall be writable (24.2.1) to the `result` output iterator. The result of the expression `val - acc` or `binary_op(val, acc)` shall be writable to the `result` output iterator. In the ranges `[first, last]` and `[result, result + (last - first)]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.

3 *Remarks*: `result` may be equal to `first`.

4 *Returns*: `result + (last - first)`.

5 *Complexity*: Exactly `(last - first) - 1` applications of the binary operation.

The only alternative to using `InputIterator`'s value type for the intermediate type that I could think of was computing the common type of the `InputIterator`'s value type and the result of the right-hand side of the repeated assignment to `acc`, e.g.

```
using it_value_type = typename iterator_traits<InputIterator>::value_type;

// The accumulator type, determined via common_type.
using acc_common_type = common_type_t<
    it_value_type
    , decltype(binary_op(it_value_type{}, *i))
    // Or acc + *i, or acc - *i, etc.
>
```

If the `InputIterator`'s value type is convertible to the result of the binary operator, but the result of the binary operator is not convertible to the `InputIterator`'s value type, then the binary operator signature we tested with `decltype` cannot be called with `acc_common_type` as its first argument:

```

struct A { };
struct B { operator A(); };

struct binary_op_type
{
    A    operator() (B, B);
};

binary_op_type binary_op;

using it_value_type = B;

using acc_common_type = common_type_t<
    it_value_type
    , decltype(binary_op(it_value_type{}, it_value_type{}))
>;

int main()
{
    binary_op(acc_common_type{}, it_value_type{}); // COMPILE FAIL.
}

```

Even worse, we could have a `binary_op` like this:

```

struct binary_op_type
{
    A    operator() (B, B);
    int  operator() (A, B);
};

```

The above `binary_op` can now be called with `acc_common_type` as the first argument, however that overload returns a different type which we did not include in our common type computation. Nor could we have, as an iterative TMP search for a common type would be dangerous in the face of potential cycles:

```

struct binary_op_type
{
    A    operator() (B, B); // New expression binary_op(A{}, B{}) to check...
    B    operator() (A, B); // New expression binary_op(B{}, B{}) to check...
};

```

This might be viable if we constrain `binary_op` in some fashion, but it is not clear to me how that could be done. More importantly, determining a common type to use for the intermediate type is likely the wrong thing to do because it means the user does not have a clear answer to the question "What type will you use to perform the accumulation?".

Since the `OutputIterator` has no value type by virtue of being an output iterator, I cannot think of any other options for the intermediate type for the algorithms without initial value parameters other than the status quo of the `InputIterator`'s value type.

There is, however, a regrettable lack of functionality with the status quo prior to C++17:

```

vector<float> f{FLT_MAX, FLT_MAX};
vector<double> d;
partial_sum(f.begin(), f.end(), back_inserter(d));
// d[1] == inf since the intermediate type is float (the value type of
// vector<float>::iterator).

```

Pre C++17, there is no way for users to specify that `partial_sum` should use a particular type as the intermediate type instead of the `InputIterator`'s value type. In C++17, there are two ways this can be done if ordered is not needed. `inclusive_scan`, the unordered counterpart of `partial_sum`, has overloads which take an initial value:

```
vector<float> f{FLT_MAX, FLT_MAX};
vector<double> d;
inclusive_scan(f.begin(), f.end(), back_inserter(d), plus<>{}, double{0.0});
// d[1] == 2 * FLT_MAX
```

or with `transform_inclusive_scan` using a transform function that converts its argument and returns the desired type:

```
vector<float> f{FLT_MAX, FLT_MAX};
vector<double> d;
transform_inclusive_scan(f.begin(), f.end(), back_inserter(d)
                        , plus<>{}
                        , [](float f) { return double{f}; });
// d[1] == 6e38F.
```

A possible post-C++17 addition of `partial_sum` signatures accepting an initial value parameter would complete this functionality.

I believe the best option is to keep the existing behavior for `<numeric>` algorithms which do not take an initial value parameter. Making any change to the type of the accumulator object would be a breaking change as `partial_sum` and `adjacent_difference` are currently specified to use accumulator objects whose type is the `InputIterator`'s value type, and none of the alternatives offer much benefit. The current behavior is clear and easy to understand.

So, I propose:

`<numeric>` algorithms which do not take an initial value parameter and initial value type template parameter will use the value type of their input iterator type as the intermediate type.

§ 3.3. Intermediate Type for Unordered `<numeric>` Algorithms

Unlike the ordered algorithms, the new unordered algorithms from the Parallelism TS v1 do not use in-order-accumulator wording. Instead, they are all specified in terms of `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` (§2.2 `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM Algorithms`).

As an example of how these definitions are used, let's take a look at `reduce` (26.8.3 [`reduce`] paragraph 3):

```
template <class InputIterator,
         class T>
    T reduce(InputIterator first, InputIterator last,
             T init);
template <class InputIterator,
         class T,
         class BinaryOperation>
    T reduce(InputIterator first, InputIterator last,
             T init,
             BinaryOperation binary_op);
```

3 Returns: `GENERALIZED_SUM(binary_op, init, *i, ...)` for every `i` in `[first, last)`

These definitions do not clearly state what the intermediate type (e.g. the return type of `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`) should be.

This is particularly scary in the face of the arbitrary reordering and partitioning that `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` allow. Consider:


```

vector<int> i{INT_MAX, INT_MAX, INT_MAX, INT_MAX};
big_int bi = reduce(d.begin(), d.end(), big_int(0));

// Possible result
// GSUM == GENERALIZED_SUM, GNSUM == GENERALIZED_NONCOMMUTATIVE_SUM
//
// bi = GSUM(operator+, big_int(0), d[0], d[1], d[2], d[3]);
//      = operator+(GNSUM(operator+, d[0], big_int(0))
//                  , GNSUM(operator+, d[1], d[2], d[3]));
//      = operator+(
//          operator+(
//              GNSUM(operator+, d[0]), GNSUM(operator+, big_int(0))
//          )
//      , operator+(
//          GNSUM(operator+, d[1], d[2]), GNSUM(operator+, d[3])
//      )
//      );
//      = operator+(
//          operator+(d[0], big_int(0))
//      , operator+(
//          operator+(GNSUM(operator+, d[1]), GNSUM(operator+, d[2])), d[3]
//      )
//      );
//      = operator+(
//          operator+(d[0], big_int(0))
//      , operator+(operator+(d[1], d[2]), d[3])
//      );
//      = ((d[0] + big_int(0)) + ((d[1] + d[2]) + d[3]));
//      = ((d[0] + big_int(0)) + ((int{INT_MAX} + int{INT_MAX}) + d[3]));
//      = ((d[0] + big_int(0)) + (int{-2} + int{INT_MAX}));
//      = ((int{INT_MAX} + big_int(0)) + int{INT_MAX - 2});
//      = (big_int(INT_MAX) + int{INT_MAX - 2});
//      = big_int(INT_MAX + INT_MAX - 2);
//
// bi = 2 * INT_MAX - 2; // Instead of 4*INT_MAX

```

The above is just one possible result that `reduce` could produce in such an example. Note that in addition to performing some of the calculations with `int` as the intermediate type, `reduce` also called `binary_op` with the initial value type as the **second** argument and the element from the sequence (whose type is the `InputIterators`'s value type) as the **first** argument. The ordered algorithms always pass intermediate objects as the first argmuent to the binary operator.

It seems sensible that we should use the same rules I suggested for the ordered algorithms (§3.4 [Intermediate Type Policy for <numeric> Algorithms](#)). This led me to the following revised definitions for `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`:

1 Define `GENERALIZED_NONCOMMUTATIVE_SUM(IntermediateType, op, a1, ..., aN)` as follows, where `IntermediateType` is a type, `op` is a binary function object (20.14 [function.object]), and `a1, ..., aN` are expressions:

- (1.1) `a1` when `N` is 1, otherwise
- (1.2) `IntermediateType(op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN)))` for any `K` where $1 < K + 1 = M \leq N$.

◆ `IntermediateType` shall be constructible from the result of the expression

```
op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN)).
```

2 Define `GENERALIZED_SUM(IntermediateType, op, a1, ..., aN)` as

`GENERALIZED_NONCOMMUTATIVE_SUM(IntermediateType, op, b1, ..., bN)` where `b1, ..., bN` may be any permutation of `a1, ..., aN`.

Note that the `N == 1` case intentionally does not convert to `IntermediateType`,

§ 3.4. Intermediate Type Policy for `<numeric>` Algorithms

In summary:

The intermediate type is the initial value type if there is an initial value, and the value type of the `InputIterator` otherwise.

§ 4. Type Requirements

Now that we have defined a clear policy for what the intermediate type should be for each `<numeric>` algorithm (§3.4 Intermediate Type Policy for `<numeric>` Algorithms), we can describe their requirements:

- Operators shall neither modify elements nor invalidate iterators or subranges in the iterator ranges specified by their parameters.
- The intermediate type shall be CopyConstructible (in-order-accumulator algorithms that take an initial value), constructible from `*first` (in-order-accumulator algorithms that do not take an initial value), or constructible from `op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN))` (`GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` algorithms). This requirement comes from the initialization of the accumulator object (in-order accumulator algorithms) or from the conversion to `IntermediateType` (`GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` algorithms).
- The right hand side of the update expression must be convertible to the intermediate type (all in-order-accumulator algorithms). This requirement comes from the update expression (the right hand side of `acc = binary_op(acc, *i)`).
- The intermediate type must be CopyAssignable (all in-order-accumulator algorithms except `adjacent_difference`) or MoveAssignable (`adjacent_difference`). This requirement comes from the update expression (the assignment in `acc = binary_op(acc, *i)`).
- The intermediate type shall be writable to the result output iterator (for algorithms which take an output range). This requirement comes from the write to the output iterator that is performed each iteration in these algorithms.

For the function objects, we've already specified the requirements for what the return type of the invocations of `binary_op` should be. We just need to require that:

- `binary_op`, `binary_op1`, `binary_op2` and `unary_op` are function objects.
- `binary_op(acc, *i)` (all in-order-accumulator algorithms), `acc + *i` (`accumulate`, `inner_product` and `partial_sum`), `acc - *i` (`adjacent_difference`), `binary_op1(acc, binary_op2(*i1, *i2))` (`inner_product`) and `acc + (*i1) * (*i2)` are required.
- The function object requirements from the new algorithms are in `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`:
 - If `a1, ..., aN` are of the same type (e.g. the homogeneous elements of an iterator range), then `op(a1, a1)` is the only signature required by `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`. This covers the new algorithms which do not take an initial value type.
 - If `a1` is of one type and `a2, ..., aN` are of the same type (e.g. an initial value `a1` and the homogeneous elements of an iterator range `a2, ..., aN`), then `op(a2, a1)`, `op(a1, a2)`, `op(a2, a2)` and `op(a1, a1)` are required by `GENERALIZED_SUM` and `op(a2, a1)`, `op(a2, a2)` and `op(a1, a1)` are required for `GENERALIZED_NONCOMMUTATIVE_SUM`. This covers the new algorithms which do take an initial value type.

While the new algorithms introduced by the Parallelism TS do have the necessary requirements forbidding modification of iterator ranges, the ranges used are not fully closed like the rest of the `<numeric>` algorithms.

There is one additional outlier case. `reduce` has a signature which only takes two `InputIterator`'s and is defined as equivalent to `return reduce(first, last, typename iterator_traits<InputIterator>::value_type{});`. This requires that `InputIterator`'s value type be

DefaultConstructible. `accumulate` does not have such a signature (by design, I believe). Ideally, I'd like to remove this signature, but I think it is too late to do so. I'd like to add the missing requirement, at least.

§ 5. Proposed Changes

The proposed changes are relative to [\[N4604\]](#), the Committee Draft for C++17. The $\diamond$ character is used to denote a placeholder section number which the editor shall choose.

Apply the following changes to 26.2 **[numerics.defns]**:

26.2 Definitions **[numerics.defns]**

1 Define `GENERALIZED_NONCOMMUTATIVE_SUM`(`IntermediateType`, `op`, `a1`, ..., `aN`) as follows, where `IntermediateType` is a type, `op` is a binary function object (20.14 **[function.object]**), and `a1`, ..., `aN` are expressions :

- (1.1) `a1` when `N` is 1, otherwise
- (1.2) `IntermediateType`(`op`(`GENERALIZED_NONCOMMUTATIVE_SUM`(`op`, `a1`, ..., `aK`), `GENERALIZED_NONCOMMUTATIVE_SUM`(`op`, `aM`, ..., `aN`))) for any `K` where $1 < K + 1 = M \leq N$.

$\diamond$ `IntermediateType` shall be constructible from the result of the expression `op`(`GENERALIZED_NONCOMMUTATIVE_SUM`(`op`, `a1`, ..., `aK`), `GENERALIZED_NONCOMMUTATIVE_SUM`(`op`, `aM`, ..., `aN`)).

2 Define `GENERALIZED_SUM`(`IntermediateType`, `op`, `a1`, ..., `aN`) as `GENERALIZED_NONCOMMUTATIVE_SUM`(`IntermediateType`, `op`, `b1`, ..., `bN`) where `b1`, ..., `bN` may be any permutation of `a1`, ..., `aN`.

Apply the following changes to 26.8 **[numeric.ops]** starting at 28.8.2 **[accumulate]**:

26.8.2 Accumulate **[accumulate]**

```
template <class InputIterator,
          class T>
    T accumulate(InputIterator first, InputIterator last,
                 T init);
template <class InputIterator,
          class T,
          class BinaryOperation>
    T accumulate(InputIterator first, InputIterator last,
                 T init,
                 BinaryOperation binary_op);
```

1 *Effects*: Computes its result by initializing the accumulator `acc`, whose type is `T`, with the initial value `init` and then modifies it with `acc = acc + *i` or `acc = binary_op(acc, *i)` for every iterator `i` in the range `[first, last)` in order.

2 *Requires*: `T` shall meet the requirements of `CopyConstructible` (Table 22) and `CopyAssignable` (Table 24) types. The result of the expression `acc + *i` or `binary_op(acc, *i)` shall be implicitly convertible to `T`. In the range `[first, last]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.

26.8.3 Reduce **[reduce]**

```
template <class InputIterator>
    typename iterator_traits<InputIterator>::value_type
    reduce(InputIterator first, InputIterator last);
```

1 *Effects*: Equivalent to: `return reduce(first, last, typename iterator_traits<InputIterator>::value_type{});`

◆ *Requires:* `InputIterator`'s value type shall satisfy the requirements of `DefaultConstructible` (Table 20).

```
template <class InputIterator,
          class T>
T reduce(InputIterator first, InputIterator last,
         T init);
```

2 *Effects:* Equivalent to: `return reduce(first, last, init, plus<>());`

```
template <class InputIterator,
          class T,
          class BinaryOperation>
T reduce(InputIterator first, InputIterator last,
         T init,
         BinaryOperation binary_op);
```

3 *Returns:* `GENERALIZED_SUM(T, binary_op, init, *i, ...)` for every `i` in `[first, last)`.

4 *Requires:* `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the range `[first, last)`.

5 *Complexity:* $O(\text{last} - \text{first})$ applications of `binary_op`.

6 *Notes:* The difference between `reduce` and `accumulate` is that `reduce` applies `binary_op` in an unspecified order, which yields a non-deterministic result for non-associative or non-commutative `binary_op` such as floating-point addition.

26.8.4 Transform reduce [`transform.reduce`]

```
template <class InputIterator, class UnaryOperation,
          class T,
          class BinaryOperation>
T transform_reduce(InputIterator first, InputIterator last,
                  UnaryOperation unary_op,
                  T init,
                  BinaryOperation binary_op);
```

1 *Returns:* `GENERALIZED_SUM(T, binary_op, init, unary_op(*i), ...)` for every `i` in `[first, last)`.

2 *Requires:* Neither `unary_op` nor `binary_op` shall invalidate subranges, or modify elements in the range `[first, last)`.

3 *Complexity:* $O(\text{last} - \text{first})$ applications each of `unary_op` and `binary_op`.

4 *Notes:* `transform_reduce` does not apply `unary_op` to `init`.

26.8.5 Inner product [`inner.product`]

```
template <class InputIterator1, class InputIterator2,
          class T>
T inner_product(InputIterator1 first1, InputIterator1 last1,
               InputIterator2 first2,
               T init);
template <class InputIterator1, class InputIterator2, class T,
          class BinaryOperation1,
          class BinaryOperation2>
T inner_product(InputIterator1 first1, InputIterator1 last1,
               InputIterator2 first2,
               T init,
               BinaryOperation1 binary_op1,
               BinaryOperation2 binary_op2);
```

1 *Effects*: Computes its result by initializing the accumulator `acc`, whose type is `T`, with the initial value `init` and then modifying it with `acc = acc + (*i1) * (*i2)` or `acc = binary_op1(acc, binary_op2(*i1, *i2))` for every iterator `i1` in the range `[first1, last1)` and iterator `i2` in the range `[first2, first2 + (last1 - first1))` in order.

2 *Requires*: `T` shall meet the requirements of `CopyConstructible` (Table 22) and `CopyAssignable` (Table 24) types. The result of the expression `acc + (*i1) * (*i2)` or `binary_op1(acc, binary_op2(*i1, *i2))` shall be implicitly convertible to `T`. In the ranges `[first1, last1]` and `[first2, first2 + (last1 - first1)]` `binary_op1` and `binary_op2` shall neither modify elements nor invalidate iterators or subranges.

26.8.6 Partial sum [`partial.sum`]

```
template <class InputIterator, class OutputIterator>
    OutputIterator partial_sum(InputIterator first, InputIterator last,
                              OutputIterator result);
template <class InputIterator, class OutputIterator,
          class BinaryOperation>
    OutputIterator partial_sum(InputIterator first, InputIterator last,
                              OutputIterator result,
                              BinaryOperation binary_op);
```

1 *Effects*: For a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, `acc` is then modified by `acc = acc + *i` or `acc = binary_op(acc, *i)` and the result is assigned to `*(result + (i - first))`.

2 *Returns*: `result + (last - first)`.

3 *Complexity*: Exactly `(last - first) - 1` applications of the binary operation.

4 *Requires*: `InputIterator`'s value type shall be `CopyAssignable` (Table 24) and constructible from the type of `*first`. The result of the expression `acc + *i` or `binary_op(acc, *i)` shall be implicitly convertible to `InputIterator`'s value type. `acc` shall be writable (24.2.1) to the `result` output iterator. In the ranges `[first, last]` and `[result, result + (last - first)]` `binary_op` shall neither modify elements nor invalidate iterators or subranges.

5 *Remarks*: `result` may be equal to `first`.

26.8.7 Exclusive scan [`exclusive.scan`]

```
template <class InputIterator, class OutputIterator,
          class T>
    OutputIterator exclusive_scan(InputIterator first, InputIterator last,
                                 OutputIterator result,
                                 T init);
```

1 *Effects*: Equivalent to: `return exclusive_scan(first, last, result, init, plus<>());`

```
template <class InputIterator, class OutputIterator,
          class T,
          class BinaryOperation>
    OutputIterator exclusive_scan(InputIterator first, InputIterator last,
                                 OutputIterator result,
                                 T init,
                                 BinaryOperation binary_op);
```

2 *Effects*: Assigns through each iterator `i` in `[result, result + (last - first))` the value of `GENERALIZED_NONCOMMUTATIVE_SUM(T, binary_op, init, *j, ...)` for every `j` in `[first, first + (i - result))`.

3 *Returns*: The end of the resulting range beginning at `result`.

4 *Requires:* `GENERALIZED_NONCOMMUTATIVE_SUM(T, binary_op, init, unary_op(*j), ...)` shall be writable (24.2.1) to the result output iterator. `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

5 *Complexity:* $O(\text{last} - \text{first})$ applications of `binary_op`.

6 *Notes:* The difference between `exclusive_scan` and `inclusive_scan` is that `exclusive_scan` excludes the *i*th input element from the *i*th sum. If `binary_op` is not mathematically associative, the behavior of `exclusive_scan` may be non-deterministic.

7 *Remarks:* `result` may be equal to `first`.

26.8.8 Inclusive scan [`inclusive.scan`]

```
template <class InputIterator, class OutputIterator>
    OutputIterator inclusive_scan(InputIterator first, InputIterator last,
                                OutputIterator result);
```

1 *Effects:* Equivalent to: `return inclusive_scan(first, last, result, plus<>());`

```
template <class InputIterator, class OutputIterator,
          class BinaryOperation>
    OutputIterator inclusive_scan(InputIterator first, InputIterator last,
                                OutputIterator result,
                                BinaryOperation binary_op);

template <class InputIterator, class OutputIterator,
          class BinaryOperation,
          class T>
    OutputIterator inclusive_scan(InputIterator first, InputIterator last,
                                OutputIterator result,
                                BinaryOperation binary_op,
                                T init);
```

2 *Effects:* Assigns through each iterator *i* in `[result, result + (last - first))` the value of

- (2.1) `GENERALIZED_NONCOMMUTATIVE_SUM(T, binary_op, init, *j, ...)` for every *j* in `[first, first + (i - result + 1))` if `init` is provided, or
- (2.2) `GENERALIZED_NONCOMMUTATIVE_SUM(typename iterator_traits<InputIterator>::value_type, binary_op, *j, ...)` for every *j* in `[first, first + (i - result + 1))` otherwise.

3 *Returns:* The end of the resulting range beginning at `result`.

4 *Requires:* `GENERALIZED_NONCOMMUTATIVE_SUM(T, binary_op, init, *j, ...)` shall be writable (24.2.1) to the result output iterator if `init` is provided; otherwise `GENERALIZED_NONCOMMUTATIVE_SUM(typename iterator_traits<InputIterator>::value_type, binary_op, *j, ...)` shall be writable to the result output iterator. `binary_op` shall not invalidate iterators or subranges, nor modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

5 *Complexity:* $O(\text{last} - \text{first})$ applications of `binary_op`.

6 *Remarks:* `result` may be equal to `first`.

7 *Notes:* The difference between `exclusive_scan` and `inclusive_scan` is that `inclusive_scan` includes the *i*th input element in the *i*th sum. If `binary_op` is not mathematically associative, the behavior of `inclusive_scan` may be non-deterministic.

26.8.9 Transform exclusive scan [`transform.exclusive.scan`]

```
template <class InputIterator, class OutputIterator,
          class UnaryOperation,
          class T,
          class BinaryOperation>
OutputIterator transform_exclusive_scan(InputIterator first, InputIterator last,
                                       OutputIterator result,
                                       UnaryOperation unary_op,
                                       T init,
                                       BinaryOperation binary_op);
```

1 *Effects*: Assigns through each iterator *i* in `[result, result + (last - first))` the value of `GENERALIZED_NONCOMMUTATIVE_SUM(T, binary_op, init, unary_op(*j), ...)` for every *j* in `[first, first + (i - result))`.

2 *Returns*: The end of the resulting range beginning at `result`.

3 *Requires*: `GENERALIZED_NONCOMMUTATIVE_SUM(T, binary_op, init, unary_op(*j), ...)` shall be [writable \(24.2.1\)](#) to the `result` output iterator. Neither `unary_op` nor `binary_op` shall invalidate iterators or subranges, or modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

4 *Complexity*: $O(\text{last} - \text{first})$ applications each of `unary_op` and `binary_op`.

5 *Remarks*: `result` may be equal to `first`.

6 *Notes*: The difference between `transform_exclusive_scan` and `transform_inclusive_scan` is that `transform_exclusive_scan` excludes the *i*th input element from the *i*th sum. If `binary_op` is not mathematically associative, the behavior of `transform_exclusive_scan` may be non-deterministic. `transform_exclusive_scan` does not apply `unary_op` to `init`.

26.8.10 Transform inclusive scan [`transform.inclusive.scan`]

```
template <class InputIterator, class OutputIterator,
          class UnaryOperation,
          class BinaryOperation>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                       OutputIterator result,
                                       UnaryOperation unary_op,
                                       BinaryOperation binary_op);

template <class InputIterator, class OutputIterator,
          class UnaryOperation,
          class BinaryOperation,
          class T>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                       OutputIterator result,
                                       UnaryOperation unary_op,
                                       BinaryOperation binary_op,
                                       T init);
```

1 *Effects*: Assigns through each iterator *i* in `[result, result + (last - first))` the value of

- (1.1) `GENERALIZED_NONCOMMUTATIVE_SUM(T, binary_op, init, unary_op(*j), ...)` for every *j* in `[first, first + (i - result + 1))` if `init` is provided, or
- (1.2) `GENERALIZED_NONCOMMUTATIVE_SUM(typename iterator_traits<InputIterator>::value_type, binary_op, unary_op(*j), ...)` for every *j* in `[first, first + (i - result + 1))` otherwise.

2 *Returns*: The end of the resulting range beginning at `result`.

3 *Requires*: `GENERALIZED_NONCOMMUTATIVE_SUM(T, binary_op, init, unary_op(*j), ...)` shall be [writable \(24.2.1\)](#) to the `result` output iterator if `init` is provided; otherwise `GENERALIZED_NONCOMMUTATIVE_SUM(typename iterator_traits<InputIterator>::value_type, binary_op, unary_op(*j), ...)` shall be [writable](#) to the `result` output iterator. Neither `unary_op` nor

`binary_op` shall invalidate iterators or subranges, or modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

4 *Complexity:* $O(\text{last} - \text{first})$ applications each of `unary_op` and `binary_op`.

5 *Remarks:* `result` may be equal to `first`.

6 *Notes:* The difference between `transform_exclusive_scan` and `transform_inclusive_scan` is that `transform_inclusive_scan` includes the *i*th input element in the *i*th sum. If `binary_op` is not mathematically associative, the behavior of `transform_inclusive_scan` may be non-deterministic. `transform_inclusive_scan` does not apply `unary_op` to `init`.

26.8.11 Adjacent difference [`adjacent.difference`]

```
template <class InputIterator,
          class OutputIterator>
    OutputIterator adjacent_difference(InputIterator first, InputIterator last,
                                      OutputIterator result);

template <class InputIterator,
          class OutputIterator,
          class BinaryOperation>
    OutputIterator adjacent_difference(InputIterator first, InputIterator last,
                                      OutputIterator result,
                                      BinaryOperation binary_op);
```

1 *Effects:* For a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, creates an object `val` whose type is `InputIterator`'s value type, initializes it with `*i`, computes `val - acc` or `binary_op(val, acc)`, assigns the result to `* (result + (i - first))`, and move assigns from `val` to `acc`.

2 *Requires:* `InputIterator`'s value type shall be `MoveAssignable` (Table 23) and shall be constructible from the type of `*first`. `acc` shall be writable (24.2.1) to the `result` output iterator. The result of the expression `val - acc` or `binary_op(val, acc)` shall be writable to the `result` output iterator. In the ranges `[first, last]` and `[result, result + (last - first)]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.

3 *Remarks:* `result` may be equal to `first`.

4 *Returns:* `result + (last - first)`.

5 *Complexity:* Exactly $(\text{last} - \text{first}) - 1$ applications of the binary operation.

§ References

§ Informative References

[N4604]

Richard Smith. [C++17 CD Ballot Document](https://wg21.link/n4604). 12 July 2016. URL: <https://wg21.link/n4604>

[P0452r0]

Bryce Adelstein LeBlach. [Binary transform_reduce\(\): The Missing Overload](https://wg21.link/p0452r0). 14 October 2016. URL: <https://wg21.link/p0452r0>